

SECTION 8 — JAVA 8+ FEATURES (Senior SDET Masterclass)

Topics Covered

- 8.1 Lambda Expressions (`invokedynamic` & `LambdaMetafactory` Mechanics)
- 8.2 Functional Interfaces & SAM (`@FunctionalInterface` Contract)
- 8.3 `Predicate<T>` (Composition with `.and()` , `.or()` , `.negate()`)
- 8.4 `Function<T, R>` (Transformation Pipelines & Chaining with `.andThen()`)
- 8.5 `Consumer<T>` (Side-Effect Execution & Chaining)
- 8.6 `Supplier<T>` (Lazy Evaluation, Test Data Hydration, & Deferred Execution)
- 8.7 Bi-Interfaces (`BiFunction` , `BiConsumer` , `BiPredicate`)
- 8.8 Method References (Static, Bound, Unbound, & Constructor References)
- 8.9 Stream API Fundamentals (Lazy Pipelines, Short-Circuiting, & Spliterators)
- 8.10 `map()` vs. `flatMap()` (Flattening Nested JSON & DTO Graphs)
- 8.11 `filter()` , 8.12 `sorted()` , `distinct()` , 8.13 `limit()` , `skip()`
- 8.14 `reduce()` (Accumulators, Associativity, & Combiners)
- 8.15 `collect()` and Advanced Collectors
- 8.16 `groupingBy()` and `partitioningBy()` (SQL-Like Test Aggregation)
- 8.17 `joining()` (Formatting Delimited Reports & Query Strings)
- 8.18 Stream vs. Traditional Loop (Performance Reality & JIT Optimizations)
- 8.19 Parallel Streams (The ForkJoinPool `commonPool` Blocking Hazard)
- 8.20 `Optional<T>` (Eager `orElse` vs. Lazy `orElseGet` Traps & Clean Usage)
- 8.21 Modern Date & Time API (`java.time` Immutability & Thread Safety)
- 8.22 Modern Java (Java 11 to 21 Features: Records, Pattern Matching, Virtual Threads)
- 8.23 High-Stakes Senior Java 8 Coding Questions & Spoken Solutions

8.1 Lambda Expressions & 8.2 Functional Interfaces

1. Theory & Core Mechanics

A Lambda expression is an anonymous function that can be passed around as a first-class citizen.

- **Single Abstract Method (SAM):** A functional interface has exactly **one abstract method**. It can have any number of `default` or `static` methods. Annotated with `@FunctionalInterface` .
- **Under the Hood (Not Anonymous Inner Classes!):**
 - Anonymous classes generate physical `$1.class` files on disk, increasing JAR size and Metaspace class-loading overhead.
 - Lambdas use the **`invokedynamic` (Indy)** bytecode instruction. At runtime, the JVM uses `LambdaMetafactory.metafactory()` to dynamically generate a lightweight call site linking directly to a synthetic private method holding the lambda's body.

Source: `(x) -> x * 2`

Bytecode:

```
0: invokedynamic #2, 0 // InvokeDynamic #0:apply:()Ljava/util/function/Function;
```

8.3 The 4 Core Functional Interfaces

Interface	Signature	Method Name	Typical Test Automation Use Case
<code>Predicate<T></code>	<code>T -> boolean</code>	<code>test(T)</code>	Filtering test cases, wait conditions
<code>Function<T, R></code>	<code>T -> R</code>	<code>apply(T)</code>	Transforming API responses, DTO mapping
<code>Consumer<T></code>	<code>T -> void</code>	<code>accept(T)</code>	Logging, taking screenshots, DB assertions
<code>Supplier<T></code>	<code>() -> T</code>	<code>get()</code>	Lazy test data generation, WebDriver

Production Code: Fluent Custom Wait Engine with Functional Interfaces

```
package com.deloitte.sdet.waits;

import java.time.Duration;
import java.time.Instant;
import java.util.function.Predicate;
import java.util.function.Supplier;

public final class FunctionalWait {

    public static <T> T waitFor(Supplier<T> stateSupplier, Predicate<T> condition, Duration timeout) {
        Instant deadline = Instant.now().plus(timeout);
        while (Instant.now().isBefore(deadline)) {
            T state = stateSupplier.get();
            if (condition.test(state)) {
                return state;
            }
            try {
                Thread.sleep(250); // Polling interval
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                throw new RuntimeException("Wait interrupted", e);
            }
        }
        throw new RuntimeException("Condition timed out after " + timeout.toSeconds() + " seconds");
    }
}
```

8.8 Method References

Shorthand syntax for lambdas that simply call an existing method (`ClassName::methodName`):

- Static Method:** `Integer::parseInt` `$equiv$ s -> Integer.parseInt(s)`
- Bound Instance Method:** `System.out::println` `$equiv$ x -> System.out.println(x)`
- Unbound Instance Method:** `String::toUpperCase` `$equiv$ (String s) -> s.toUpperCase()`

4. **Constructor Reference:** `ArrayList::new $equiv$ () -> new ArrayList<>()`

8.9 Stream API Fundamentals & 8.10 `map()` vs. `flatMap()`

A Stream is a sequence of elements supporting sequential and parallel aggregate operations:

- **Lazy Evaluation:** Intermediate operations (`filter` , `map` , `sorted`) are never executed until a terminal operation (`collect` , `count` , `findFirst`) is invoked.
- **Single-Use:** A stream cannot be reused once a terminal operation has executed.

`map()` vs. `flatMap()` (The 1-to-1 vs. 1-to-Many Transformation)

- **`map(Function<T, R>)`** : Transforms each element of type `T` into a single element of type `R` (1-to-1).
- **`flatMap(Function<T, Stream<R>>)`** : Transforms each element of type `T` into a stream of elements, and **flattens** multiple nested streams into a single composite stream (1-to-Many). Essential for parsing nested arrays in JSON payloads!

```
// Parsing nested JSON response: Order -> List<OrderItem>
List<Order> orders = fetchOrdersFromApi();

// map(): Returns List<List<OrderItem>> (Nested, difficult to assert)
List<List<OrderItem>> nestedItems = orders.stream().map(Order::items).toList();

// flatMap(): Returns List<OrderItem> (Flattened single list of all items across all orders)
List<OrderItem> allItems = orders.stream()
    .flatMap(order -> order.items().stream())
    .toList();
```

8.15 `collect()` and Advanced `collectors`

8.16 `groupingBy()` and `partitioningBy()`

- **`groupingBy()`** : Analogous to SQL `GROUP BY` . Partitions items into a `Map<K, List<T>>` based on a classifier function.
- **`partitioningBy()`** : Special case of `groupingBy` returning a `Map<Boolean, List<T>>` splitting elements strictly into `true` and `false` groups.

Production Code: Aggregating API Test Results

```
package com.deloitte.sdet.reporting;

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public record ApiTestResult(String endpoint, int statusCode, long responseTimeMs, String status)

public class TestAnalyticsEngine {
```

```

public static void analyzeSuite(List<ApiTestResult> results) {
    // 1. Group tests by HTTP Status Code
    Map<Integer, List<ApiTestResult>> byStatusCode = results.stream()
        .collect(Collectors.groupingBy(ApiTestResult::statusCode));

    // 2. Count failures per endpoint
    Map<String, Long> failuresPerEndpoint = results.stream()
        .filter(r -> r.statusCode() >= 400)
        .collect(Collectors.groupingBy(ApiTestResult::endpoint, Collectors.counting()));

    // 3. Partition by SLA compliance (Response time <= 1000ms)
    Map<Boolean, List<ApiTestResult>> slaPartition = results.stream()
        .collect(Collectors.partitioningBy(r -> r.responseTimeMs() <= 1000));

    // 4. Join failed endpoint names into comma-delimited string
    String failedEndpoints = results.stream()
        .filter(r -> r.statusCode() >= 500)
        .map(ApiTestResult::endpoint)
        .distinct()
        .collect(Collectors.joining(", ", "[", "]"));
}
}

```

8.18 Stream vs. Traditional Loop (Performance Reality)

- **Simple Primitives over Small Arrays (\$N < 10,000\$):** Traditional `for` loop is approximately **2x to 3x faster** than a Stream due to zero object allocation and native CPU JIT loop unrolling.
- **Large Collections (\$N > 100,000\$):** Performance difference is negligible.
- **The Verdict:** Write Streams for readability, declarative clarity, and maintenance. Use traditional loops in ultra-hot low-latency loops or when primitive performance is critical.

8.19 Parallel Streams: The ForkJoinPool Hazard

[!CAUTION]

Never use `.parallelStream()` for API or UI test execution!

Why Parallel Streams Collapse Enterprise Test Suites

1. All parallel streams across the entire JVM share a **single, global, JVM-wide `ForkJoinPool.commonPool()`**.
2. The common pool defaults to `Runtime.getRuntime().availableProcessors() - 1` worker threads (e.g. 3 threads on a 4-core CI container).
3. `ForkJoinPool` is designed exclusively for **CPU-bound, non-blocking calculations** (e.g. matrix math).
4. Automated tests involve **blocking I/O operations** (waiting for HTTP network responses, Selenium socket commands).

5. If one test thread makes a blocking HTTP call inside a parallel stream, it locks that worker thread. Three concurrent tests will starve the entire JVM's common pool, causing all other parallel streams across the application to deadlock or grind to a halt!
6. **Senior Resolution:** Use dedicated, bounded `ExecutorService` thread pools for parallel test execution.

8.20 `Optional<T>`

`Optional<T>` is a container object designed specifically as a **method return type** to communicate the potential absence of a value without returning `null`.

The `orElse` vs. `orElseGet` Performance Trap

```
// CATASTROPHIC ANTI-PATTERN: orElse() is evaluated EAGERLY!
// generateAuthToken() executes on EVERY call, even if cachedToken is present!
String token = cachedToken.orElse(generateAuthToken());

// OPTIMAL SENIOR PATTERN: orElseGet() is evaluated LAZILY via Supplier!
// generateAuthToken() ONLY executes if cachedToken is empty!
String token = cachedToken.orElseGet(() -> generateAuthToken());
```

8.21 Modern Date & Time API (`java.time`)

Prior to Java 8, `java.util.Date` and `java.text.SimpleDateFormat` were notoriously **mutable and NOT thread-safe**. Sharing a `SimpleDateFormat` across parallel TestNG threads caused corrupted date parsing and random exceptions.

Java 8 introduced the immutable, thread-safe `java.time` package:

- **Instant** : Represents a machine timestamp in UTC (epoch seconds/nanos).
- **LocalDate** , **LocalTime** , **LocalDateTime** : Human-readable dates without timezones.
- **ZonedDateTime** : Full date-time with timezone offset and daylight saving rules.
- **Duration** : Time-based amount (seconds, millis) for timeouts.
- **DateTimeFormatter** : 100% thread-safe date parsing and formatting.

```
// Thread-safe dynamic test data timestamp generation
String runId = DateTimeFormatter.ofPattern("yyyyMMdd_HHmss")
    .withZone(ZoneId.of("UTC"))
    .format(Instant.now());
```

8.22 Modern Java (Java 11 to 21 Features for SDETs)

Feature	Version	Value to Senior SDET Frameworks
<code>`var`</code>	Java 10/11	Local variable type inference; cleans up verbose types

Text Blocks <code>"""</code>	Java 15	Multi-line JSON/SQL payloads without escaped quotes
<code>`record`</code>	Java 16	Immutable DTOs with auto <code>`equals`</code> , <code>`hashCode`</code> , getters
Pattern Matching	Java 16/21	<code>`if (obj instanceof LoginPage lp)`</code> & switch patterns
Sealed Classes	Java 17	Restricts inheritance hierarchy to permitted subclasses
Virtual Threads	Java 21	Lightweight threads (Loom) for high-scale API testing

8.23 High-Stakes Senior Java 8 Coding Questions & Spoken Solutions

Q1: "Find the second highest salary from a list of Employee DTOs using Java 8 Streams, handling duplicates and nulls."

```
public static Optional<Double> findSecondHighestSalary(List<Employee> employees) {
    if (employees == null || employees.isEmpty()) {
        return Optional.empty();
    }

    return employees.stream()
        .filter(Objects::nonNull)
        .map(Employee::salary)
        .filter(Objects::nonNull)
        .distinct() // Remove duplicate identical salary tiers
        .sorted(Comparator.reverseOrder()) // Sort descending
        .skip(1) // Skip the top salary
        .findFirst(); // Retrieve second highest
}
```

Q2: "Group a list of test error logs by error code and return only the top 3 most frequent error codes."

```
public static List<String> getTop3ErrorCodes(List<String> rawErrorLogs) {
    return rawErrorLogs.stream()
        .filter(Objects::nonNull)
        .collect(Collectors.groupingBy(log -> log, Collectors.counting())) // Frequency map
        .entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed()) // Sort by frequency desc
        .limit(3)
        .map(Map.Entry::getKey)
        .toList();
}
```